Studio Nx

AI 구현 설명 33 — P1-03 백엔드 공통

2026-10-12

AI 구현 설명 33 — P1-03 백엔드 공통 (오류 응답 · 토큰 검사 · 로그인 사용자 · ORM 모델)

항목	내용
작성	Claude (AI), 2026-10-06
작업	P1-03 백엔드 공통: PRD 01 FR-105, FR-108, FR-112, FR-122
브랜치	`feature/phase1/p1-03-backend-core`
바꾼 파일	`studionx/backend/app/core/errors.py`, `core/security.py`, `api/deps.py`, `models/user.py`, `models/inquiry.py`, `models/project.py`, `models/track.py`
진행 방식	**함수 하나씩** 구현 → 사용자 검토 → 다음 (2026-10-06 규칙)
함수별 설명	`docs/ai-notes/backend(fastAPI_Python)/P1-03 - 백엔드 공통/01 ~ 13` (md, PDF), 합본 backend(fastAPI Python)/P1-03 - 백엔드 공통/P1-03 backend core.pdf
이전 문서	32 p1-02 frontend api core.md

1. 한 일

나머지 API(P1-04 이후)가 모두 쓰는 백엔드 바탕을 만들었습니다.

부분	역할
오류 응답 (`core/errors.py`)	어떤 오류든 `{"error": {code, message, details}}` 한 모양의 JSON 으로 응답
토큰 검사 (`core/security.py`)	Access Token 이 진짜, 유효한지 확인하고 회원 번호를 꺼냄
로그인 사용자 (`api/deps.py`)	라우터에 `user: CurrentUser`, `user: OptionalUser` 로 회원을 넣어 줌
ORM 모델 (`models/`)	MySQL 테이블 5개를 파이썬 클래스로: 칸, 기본값, FK, 관계

2. 항목 13개

#	파일	항목	하는 일	설명
01	`errors.py`	`app_error_handler()`	코드가 일부러 던진 `AppError` → 오류 JSON	01
02	`errors.py`	`validation_error_handler()`	입력 형식 오류 → 422 + 칸별 `details`	02
03	`errors.py`	`http_error_handler()`	404, 405, 413 등 기본 오류 → 같은 모양	03
04	`errors.py`	`unexpected_error_handler()`	예상 못 한 오류 → 500 일반 문장 (내용은 로그, Sentry 에만)	04
05	`security.py`	`decode_access_token()`	JWT 서명, 만료, 종류 확인 → `user_id`	05
06	`deps.py`	`get_current_user()`	토큰 → DB 의 회원 → `ACTIVE` 확인. 아니면 401	06
07	`deps.py`	`get_current_user_optional()`	06 과 같지만 실패하면 `None` (손님)	07
08	`models/user.py`	`User`	회원 15칸, `UserRole`, `UserStatus`	08
09	`models/inquiry.py`	`Inquiry`	문의 12칸, `InquiryStatus`, FK(회원, 비어도 됨)	09

#	파일	항목	하는 일	설명
10	`models/project.py`	`Project`	프로젝트 17칸, `Decimal` BPM, JSON 설정	10
11	`models/project.py`	`ProjectMember`	멤버, 복합 기본 키, `MemberRole`, `ON DELETE CASCADE`	11
12	`models/track.py`	`Track`	트랙 Phase 1 칸만, `TrackType`	12
13	`project.py`, `track.py`	관계	`project.members`, `project.tracks` (순서, 함께 저장)	13

3. 함께 움직이는 모습

로그인한 회원이 대시보드를 열 때 (`GET /api/v1/me/dashboard`)

순서	항목	결과
①	브라우저 `api.js` (JS 02)	`Authorization: Bearer eyJ…` 를 붙여 보냄
②	06 `get_current_user()` → 05 `decode_access_token()`	토큰 확인 → `7`
③	06 → 08 `User`	`db.get(User, 7)` → `status` 가 `ACTIVE`
④	라우터 (P1-16)	`user` 로 7번 회원을 받아 응답

토큰이 만료되었을 때

순서	항목	결과
①	05	`AppError(UNAUTHORIZED, 401, "Please sign in again.")`
②	01	401 `{"error": {"code": "UNAUTHORIZED", …}}`
③	브라우저 `api.js`	재발급(P1-05) 후 같은 요청을 한 번 다시 보냄

오류가 화면까지 가는 길

오류	처리기	응답	화면 (2026-10-06 결정)
회원가입 이메일 중복 등 (`AppError`)	01	409 등 + 문구	폼 아래 문구
입력 형식 틀림	02	422 + 칸별 `details`	칸 아래 문구
없는 API, 권한 없음	03	404, 403	페이지 주 데이터면 `errors/<상태>.html`
서버 고장	04	500 일반 문장	페이지 주 데이터면 `errors/500.html`, 버튼이면 문구

4. 모델 정리

모델	테이블	칸	Enum	FK
`User`	`users`	15	`UserRole`, `UserStatus`	—
`Inquiry`	`inquiries`	12	`InquiryStatus`	`user_id` → users (비어도 됨)
`Project`	`projects`	17	—	`owner_id` → users (`template_id` 는 Phase 2)
`ProjectMember`	`project_members`	5	`MemberRole`	`project_id` (CASCADE), `user_id`
`Track`	`tracks`	5 / 19	`TrackType`	`project_id` (CASCADE)

모든 모델에 공통으로 적용한 규칙:

- 테이블은 SQL 파일(`studionx/db/`)이 만듭니다. 모델은 그 모양을 따라 적기만 합니다.
- 기본값은 `server_default` 로 DB 에 맡깁니다. 기본값의 원본은 SQL 파일 하나뿐입니다.
- 시각은 `DATETIME(3)` 이고 UTC 로 저장합니다.
- `role`, `status` 같은 ENUM 칸은 `StrEnum` 으로 만듭니다. 그래서 `UserStatus.ACTIVE == "ACTIVE"` 가 참입니다.
- 검색용 색인은 모델에 적지 않습니다. DB 에만 있으면 됩니다.

5. 보안 정리

항목	한 일
오류 내용 숨김 (04)	500 응답에 SQL, 경로, 비밀값이 들어가지 않음. 내용은 터미널 로그, Sentry 에만
JWT 방식 고정 (05)	`HS256` 만 받음 → 서명 없는 `alg: none` 위조 차단
토큰 종류 확인 (05)	리프레시 토큰(3일)을 Access Token 자리에 못 씀
같은 401, 같은 문구 (05, 06)	만료, 위조, 없는 회원, 정지, 탈퇴를 구별해 알려 주지 않음
DB 재확인 (06)	30분 토큰이 남아 있어도 탈퇴, 정지 회원은 바로 막힘 (FR-108)
비밀값	`JWT_SECRET` 은 `.env` 에만 (64바이트). 값은 출력하지 않고 길이만 확인

6. 템플릿과 달라진 점

항목	템플릿	구현	이유
`ErrorCode`	—	`BAD_REQUEST`(400), `METHOD_NOT_ALLOWED`(405) 추가	03 이 기본 오류를 모두 같은 모양으로 바꾸려면 필요
`decode_access_token`	한 함수	검사를 `_decode_token(token, type)` 으로 분리	P1-05 `decode_refresh_token` 이 한 줄이 됨
`Project.template_id`	FK	FK 없이 칸만	가리킬 `project_templates` 모델이 Phase 2
`Track`	Phase 1 칸	그대로 (3칸)	나머지 14칸은 시퀀서(Phase 2)에서
`project.tracks`	`order_by` 만	`cascade="all, delete-orphan"`, `passive_deletes` 추가	트랙을 읽은 상태로 프로젝트를 지울 때 `NOT NULL` 오류 방지

7. 시험

파일	대상	시험 수	방식
`tests/test_errors.py`	01 ~ 04	12	시험용 앱 + 실제 앱 (없는 API → JSON 404, 없는 페이지 → HTML 404)
`tests/test_security.py`	05	9	`jwt.encode` 로 만든 토큰 (만료, 위조, `alg: none` 등)
`tests/test_deps.py`	06, 07	13	가짜 DB 세션 (`FakeDb`)
`tests/test_user_model.py`	08	4	실제 MySQL (06 도 실제 DB 로 확인)
`tests/test_inquiry_model.py`	09	5	실제 MySQL
`tests/test_project_model.py`	10	4	실제 MySQL

파일	대상	시험 수	방식
`tests/test_project_member_model.py`	11	5	실제 MySQL
`tests/test_track_model.py`	12	4	실제 MySQL
`tests/test_model_relationships.py`	13	6	실제 MySQL

- `tests/conftest.py` 에 공통 도구를 두었습니다.
  - `db`: 실제 DB 세션. 연결할 수 없으면 시험을 건너뛰고, 넣은 행은 시험이 끝나면 되돌린다(rollback).
  - `assert_matches_db(모델)`: 모델의 칸, `NULL` 허용, FK(`ON DELETE` 포함)를 실제 테이블과 비교합니다.
- 전체 시험 76개가 통과했고, `ruff` 검사도 통과했습니다.
- 시험이 끝난 뒤 DB 에 남은 시험 행은 0개였습니다.

8. 다음 작업이 쓰는 곳

작업	쓰는 것
P1-04 회원가입	`User`, `AppError`(409 중복, 422 규칙), 02 의 422
P1-05 로그인, 재발급, 로그아웃	`_decode_token(…, "refresh")`, `User.status`, `last_login_at`
P1-08, 09 회원 정보, 탈퇴	`CurrentUser`, `UserStatus.DELETED`, `deleted_at`
P1-12 문의	`OptionalUser`, `Inquiry`
P1-13 프로젝트	`CurrentUser`, `Project`, `project.members`, `project.tracks` (한 번에 저장)
P1-16 대시보드	`CurrentUser`, `storage_*_bytes`
P1-22 오류 페이지	`register_exception_handlers()` 의 TODO: 페이지 요청이면 오류 HTML